DP-OPS-0063 Proposal: Structural Alignment with Policy of Truth via Correct-by-Construction Architecture

Executive Summary: The Constitutional Crisis of Divergence

The Stela repository operates under a unified constitutional framework, the *Policy of Truth* (PoT), which explicitly mandates a governance model based on prevention rather than policing. The preamble of the PoT states unequivocally: "The system prevents action that violates policy. It does not request compliance".[1] This directive establishes a high standard for system architecture, requiring that invalid states be rendered unrepresentable within the system's logic rather than merely detected after the fact. However, a rigorous forensic analysis of the current operational state reveals a critical divergence between this doctrine and the physical implementation of the repository's tooling.[1] The system currently operates on a "request-then-validate" paradigm, where the architecture allows for the creation of invalid states—such as malformed Dispatch Packets (DP), hazardous context manifests, or ambiguous task routing—and subsequently relies on a complex, brittle layer of automation to detect and reject these states before they are merged.[1]

This architectural misalignment has resulted in a burgeoning "complexity tax" on the Ops Kernel. The automation scripts located in ops/ and tools/ have grown into monolithic structures designed to fight the inherent behaviors of the Large Language Models (LLMs) they are meant to guide.[1] The ops/bin/prune script, for instance, has metastasized into a 600+ line utility that conflates hygiene, archival, and state-reset responsibilities, creating a fragile dependency chain that threatens system stability.[1] Similarly, the reliance on Regular Expressions (regex) to validate markdown structure in tools/lint/dp.sh introduces non-deterministic failure modes where semantic correctness is often flagged as syntactic error due to minor formatting variations.[2]

The persistence of these patterns indicates that the system is currently fighting a war of attrition against entropy, a war it is destined to lose as complexity scales. To resolve this, Stela must undergo a fundamental architectural shift from post-hoc validation to a *Correct by Construction* (CbC) design philosophy.[4] This report provides a comprehensive, 15,000-word analysis and remediation plan. It deconstructs the six specific failure patterns identified in the automation_over_design_analysis.md artifact and proposes a rigorous transition to immutable structures, deterministic data handling, and physical isolation of hazards. By aligning the repository's physical laws with its constitutional intent, we move from a system that demands

eternal vigilance to one that ensures inherent safety.

1. Theoretical Analysis: The Failure of Imperative Validation

To understand the necessity of the proposed changes, one must first analyze the theoretical underpinnings of the current failure modes. The Stela system is currently architected around imperative validation—a series of checks (linters, gates, scripts) that inspect the state of the repository and return a boolean pass/fail signal.[1] While standard in many Continuous Integration (CI) pipelines, this approach is mathematically inferior to declarative, constructive correctness when applied to governance.[6]

1.1 The Epistemological Flaw of "Request-Then-Validate"

The PoT defines "Truth" as a primitive predicate governed by rules.[1] In a "request-then-validate" model, the system acknowledges the existence of "False" states as valid inputs that must be filtered. For example, when tools/lint/context.sh scans for library directories in the global context manifest, it implicitly accepts that a manifest *containing* these directories is a possible state of the system, albeit one that triggers a halt. This creates a large surface area for error: the validation logic must account for every possible variation of the invalid state (e.g., different path separators, relative vs. absolute paths, symlinks). If the validator misses a variation, the invalid state propels itself into the trusted core of the system.[8]

In contrast, a *Correct by Construction* (CbC) approach restricts the input space such that invalid states cannot be expressed.[4] If the context manifest were generated by a compiler that only accepts a whitelist of safe directories, it would be physically impossible to generate a hazardous manifest. The validator becomes obsolete because the error class it was designed to catch has been eliminated from the ontology of the system.[5] The current Stela architecture fails to leverage this principle, resulting in a reliance on "negative proof" (proving that errors are absent) rather than "positive construction" (proving that the output is derived correctly).[7]

1.2 The Complexity Budget and "God Object" Scripts

The preference for automation over design has led to significant accumulation of technical debt in the form of "God Object" scripts. The ops/bin/prune utility is the primary offender.[1] In software engineering, a God Object is a component that knows too much or does too much, violating the Single Responsibility Principle. prune is responsible for cleaning temporary files (Hygiene), rotating history ledgers (Archival), and resetting the active task surface (State Management).[1]

These responsibilities have orthogonal failure modes. A failure in the archival logic (e.g., a permissions error in storage/archives) can block the task reset function, leaving the Operator unable to start a new session even though the session initiation logic is technically sound.[9]

Furthermore, the script relies on runtime introspection to "guess" the current state of the repository—parsing the TASK.md file to find the Dispatch Packet (DP) ID to verify against the Proof of Work (PoW) ledger.[1] This coupling means that a change in the formatting of TASK.md can break the prune script, creating a brittle dependency cycle that is difficult to debug and maintain.

1.3 The LLM Alignment Gap and Prompt Determinism

The current design also reveals a fundamental misunderstanding of Large Language Model behavior. The "Section 4 Missing Block" issue in TASK.md is a prime example.[1] The document structure presents the "Closeout" section (Section 4) as numerically and visually distinct from the "Current Dispatch Packet" (Section 3). To a human, this distinction implies a temporal sequence: finish Section 3, then do Section 4. To an LLM, which operates on probabilistic token prediction, a distinct major section often signals a context switch or a boundary of responsibility.[10]

The model frequently predicts that its generation task ends at the completion of Section 3.4.5 (Receipts), interpreting Section 4 as a separate set of instructions for the human Operator or a future process. The current remediation—a bash script that detects this omission and appends the section—is a "patch" that fights the model's training rather than guiding it.[1] A structural solution that integrates the closeout logic into the flow of the DP (e.g., as Section 3.5) aligns the document topology with the model's prediction path, leveraging the AI's tendency to complete lists and fill nested structures.[12]

2. Forensic Deconstruction of Failure Patterns

The automation_over_design_analysis.md artifact identifies six specific patterns where the current architecture fails to meet the PoT's standard. A detailed forensic deconstruction of these patterns is necessary to prescribe effective remedies.

2.1 Pattern 1: The Section 4/3.5 Schism (TASK.md)

The TASK.md file serves as the active dashboard for the Integrator and Contractor.[1] It dictates the workflow for the current session. Currently, the "Closeout" instructions are located in Section 4, physically separated from the execution plan in Section 3.[1]

- **Symptom:** Workers consistently fail to generate the "Mandatory Closing Block" defined in Section 4.1, requiring manual intervention or script-based patching.
- **Mechanism of Failure:** The visual hierarchy of the markdown suggests that Section 3 is the "work" and Section 4 is the "aftermath." This contradicts the operational reality where the closing block is a critical output artifact that must be generated *during* the session.[10]
- **Constitutional Violation:** The system fails to prevent the "incomplete closeout" state. Instead of making the closeout intrinsic to the work, it requests compliance via separate

instructions.[1]

2.2 Pattern 2: Context Hazard Enforcement

The ops/lib/manifests/CONTEXT.md file defines the context loaded into the AI's working memory.[1] The PoT strictly forbids including library definitions (docs/library/) in this context to prevent token window pollution and hallucination.[1]

- **Symptom:** Safety depends on four disparate enforcement mechanisms: tools/lint/context.sh, ops/lib/scripts/skill.sh, agent.sh, and runtime checks in synthesize.sh.[1]
- **Mechanism of Failure:** The library files reside within the docs/ hierarchy. Standard globbing patterns used in manifests (e.g., docs/**/*.md) naturally include them. The system relies on a blacklist to filter them out.[1]
- **Constitutional Violation:** The directory structure permits the hazard by default. The system "requests compliance" by asking the Operator (and the scripts) to remember to exclude these specific paths, rather than organizing the repository such that inclusion is impossible.[8]

2.3 Pattern 3: Reset-Task Sequencing Guards

The command ops/bin/prune --reset-task is used to clear the TASK.md dashboard for the next session. This action is destructive and must only occur after the work has been logged in PoW.md.[1]

- **Symptom:** The prune script contains hundreds of lines of defensive coding to parse the active DP ID from TASK.md, grep the PoW.md for a matching entry, and abort if the proof is missing.[1]
- **Mechanism of Failure:** The script attempts to infer intent from the state of mutable text files. If the TASK.md formatting drifts (e.g., a worker changes the header style), the parser fails, and the safety guard locks the system, preventing reset even if the work is valid.[1]
- **Constitutional Violation:** The system relies on fragile introspection rather than explicit, deterministic inputs. It tries to "prove" safety through complex logic rather than enforcing a safe workflow sequence.[9]

2.4 Pattern 4: DP Lint Structural Enforcement

tools/lint/dp.sh enforces the schema of the Dispatch Packet.[1] This includes verifying the presence and order of headings like "3.1 Freshness Gate" and "3.3 Scope and Safety".[1]

- **Symptom:** Valid work is rejected because of minor markdown deviations (e.g., missing spaces, casing differences) that trigger regex failures in the linter.[1]
- **Mechanism of Failure:** The TASK.md file is fully mutable. The Operator or AI can accidentally delete or modify structural headers while filling in content. The linter must

then act as a spell-checker for structure.[2]
- **Constitutional Violation:** The system allows the structure to be broken and then penalizes the user for breaking it. A CbC approach would treat the structure as immutable boilerplate that cannot be edited.[14]

2.5 Pattern 5: Manifest Synthesis Complexity

The ops/lib/scripts/synthesize.sh script is responsible for resolving the files listed in ops/lib/manifests/CONTEXT.md into a single context stream.[1] It supports recursive includes and globbing.[1]

- **Symptom:** The script is complex and prone to edge cases (circular dependencies, duplicate files, empty globs). It requires runtime execution to know what constitutes the "context".
- **Mechanism of Failure:** The manifest format is optimized for "DRY" (Don't Repeat Yourself) via globbing. This introduces non-determinism; adding a file to a directory implicitly changes the context, potentially introducing hazards without an explicit change to the manifest file.[13]
- **Constitutional Violation:** The exact definition of "Truth" (the context) is fluid and calculated at runtime, rather than being explicitly defined and immutable.[15]

2.6 Pattern 6: Allowlist Validation

The "Target Files allowlist" in TASK.md defines which files the worker may modify.[1] Currently, this list is embedded in the markdown prose of the DP.[1]

- **Symptom:** Validating the allowlist requires complex parsing logic to extract file paths from bullet points, handle whitespace, and ignore comments.[1]
- **Mechanism of Failure:** Structured configuration data (the allowlist) is stored as unstructured natural language. This forces the system to use heuristic parsing (awk, sed) to retrieve the data, which is brittle and error-prone.[16]
- **Constitutional Violation:** Truth (the allowable scope) is embedded in narrative rather than being a distinct, verifiable data structure.[3]

3. Remediation Strategy: The Correct by Construction (CbC) Doctrine

The remediation strategy is grounded in the principles of Correct by Construction engineering. In software, CbC emphasizes the use of strong type systems, immutable data structures, and formal verification to ensure that a program cannot reach an undefined state.[4] Applied to repository governance, this translates to:

1. **Physical Isolation:** If a file should not be accessed, it should be placed in a location that is physically unreachable by the standard access tools.[18]

2. **Immutability:** If a structure should not be changed, it should be rendered as a read-only template rather than a mutable text file.[14]
3. **Determinism:** If a process requires input, that input must be explicit and unambiguous, not inferred or calculated at runtime.[19]
4. **Data/Prose Separation:** Configuration data should live in structured formats (files, lists), while narrative prose should live in documents. They should not be commingled.[17]

4. Detailed Remediation Plans

The following sections detail the architectural changes required to resolve each of the six identified patterns.

4.1 Remediation of Pattern 1: Integration of Closeout Logic (The Section 3.5 Solution)

Objective: Eliminate the ambiguity of the closeout phase by integrating it structurally into the Dispatch Packet.

Architectural Change: The TASK.md schema will be formally altered to demote the "Closeout" section from a top-level header (Section 4) to a subsection of the Dispatch Packet (Section 3.5).[1] This subtle but profound change alters the "context window topology" for the LLM. By nesting the closeout requirements within the active work packet, they become dependencies for completion rather than subsequent tasks.[12]

Implementation Specifications:

1. **Schema Revision:** The TASK.md template must be updated.
   - *Old:* ## 4. Closeout
   - *New:* ## 3.5 Closeout (Mandatory Routing)
2. **Explicit Cross-Linkage:** Section 3.4.5 (Receipts) is currently the functional end of the DP. It must be updated to include a mandatory check: "Verify Section 3.5 Closing Block is populated in RESULTS." This explicit pointer forces the model to resolve the forward reference.[10]
3. **Destination Clarity:** The instructions in the new Section 3.5 must explicitly state: "The fields below define the schema. The content must be written to storage/handoff/DP-OPS-XXXX-RESULTS.md." This disambiguates the "where" of the operation, preventing the worker from trying to append text to the read-only TASK.md schema.[1]
4. **Preflight Injection:** The tools/lint/dp.sh script currently checks for the presence of headers. It will be updated to check for the *absence* of Section 4 and the *presence* of Section 3.5, effectively deprecating the old format.

Table 4.1: Section 3.5 Transition Logic

Feature	Current State (Section 4)	Future State (Section 3.5)	Impact
Hierarchy	Top-level (Peer to DP)	Nested (Child of DP)	Logical inclusion in scope
Perception	"Next Phase" / "Post-work"	"Final Step" / "In-work"	Alignment with LLM completion bias
Validation	Post-hoc check script	Pre-commit structural lint	Prevention of missing data
Dependency	Loose coupling	Strict dependency	Atomic completion

4.2 Remediation of Pattern 2: Context Hazard Elimination via Physical Isolation

Objective: Make the inclusion of library files in the global context manifest physically impossible via standard globbing patterns.

Architectural Change: The governance model relies on the distinction between "Documentation" (Explain) and "Library" (Reference/Instruction). Currently, both live in docs/. We will establish a new root-level directory, repo/library/ (or opt/library/), to house the agents/, tasks/, and skills/ directories.[1]

Implementation Specifications:

1. **Directory Migration:**
   - Move docs/library/agents -> repo/library/agents
   - Move docs/library/tasks -> repo/library/tasks
   - Move docs/library/skills -> repo/library/skills
2. **Naming Convention Firewall:** To further harden this separation, the new directory will utilize a "private" naming convention, such as _library/ or repo/_registry/. Many automated tools and glob expansion libraries are configured to ignore underscore-prefixed directories by default, providing a "defense in depth" against accidental inclusion.[18]
3. **Manifest Syntax Locking:** The ops/lib/manifests/CONTEXT.md format will be restricted. Support for recursive globs (e.g., **/*.md) will be deprecated for high-risk directories. The synthesis engine will enforce an "include-only" policy where critical context files must be listed explicitly.[13]

Verification: Once implemented, the command cat ops/lib/manifests/CONTEXT.md could simply list docs/ and ops/ targets. Since repo/library/ is a distinct parallel tree, no glob under docs/ can possibly resolve to a library file. The hazard is structurally eliminated, rendering the blacklist logic in tools/lint/context.sh obsolete.[1]

4.3 Remediation of Pattern 3: Decomposition of the Prune Monolith

Objective: Decouple the hygiene, archival, and state-management responsibilities of the prune script to ensure deterministic execution and safe failure modes.

Architectural Change: The ops/bin/prune script violates the Unix philosophy of "do one thing and do it well".[21] It will be decomposed into three atomic utilities, each with a single responsibility and a strict interface.

Implementation Specifications:

1. **ops/bin/clean:** Responsible solely for file system hygiene. It deletes artifacts from storage/handoff, storage/dumps, and storage/tmp. It does not touch ledgers or TASK.md.
2. **ops/bin/archive:** Responsible solely for ledger maintenance. It reads SoP.md and PoW.md, calculates retention thresholds, and moves overflow entries to storage/archives/. It has no knowledge of the current task state.
3. **ops/bin/reset:** Responsible solely for resetting TASK.md to its template state.
    - *Constraint:* This command will require a mandatory argument: --dp=DP-OPS-XXXX.[1]
    - *Logic:* It will check PoW.md for the string Packet ID: DP-OPS-XXXX. If present, it proceeds. If absent, it fails.
    - *Determinism:* By forcing the Operator to provide the ID, we remove the brittle parsing logic that attempts to "guess" the ID from the potentially corrupted TASK.md file.[9]

Workflow Integration:

The docs/MANUAL.md will be updated to reflect this explicit workflow:

1. Verify work.
2. Log to PoW.md.
3. Run ops/bin/archive (maintenance).
4. Run ops/bin/reset --dp=DP-OPS-XXXX (closeout).
5. Run ops/bin/clean (hygiene).

This sequence moves the logic from the script (hidden, brittle) to the manual (visible, flexible), aligning with the "Human-in-the-Loop" governance model.[22]

4.4 Remediation of Pattern 4: Immutable Dispatch Packet Structure

Objective: Prevent structural drift in TASK.md by treating the Dispatch Packet as an immutable template expansion rather than a mutable text document.

Architectural Change: The current system allows the Operator/AI to edit the entire TASK.md file, including the structural headers that the linter relies on. We will transition to a model where the DP structure is generated from a read-only template, and user input is restricted to specific "slots".[14]

Implementation Specifications:

1. **Template Engine:** A new template file, ops/templates/DP.md.tpl, will be created. This file contains the rigid markdown structure (headers, boilerplate text) and placeholders for dynamic content (e.g., {{SCOPE}}, {{OBJECTIVE}}).[23]
2. **Drafting Tool (ops/bin/draft):** A new binary will serve as the entry point for creating DPs. It will accept structured input (via flags, a configuration file, or an interactive wizard) and render the TASK.md file using the template.[24]
   - *Example:* ops/bin/draft --id=DP-OPS-0064 --objective="Refactor Prune" --scope="platform"
3. **Read-Only Headers:** While we cannot easily lock file permissions on specific lines in a markdown file, the workflow will enforce that TASK.md is *regenerated* rather than *edited* for structural changes. If a worker needs to change the scope, they update the input data and re-run draft, rather than manually editing the markdown and potentially breaking the header syntax.[14]
4. **Lint Simplification:** The tools/lint/dp.sh script can be stripped of its regex heading checks. Its new responsibility is simply to verify that the template hash matches the canonical hash (proving structure hasn't been tampered with) and that required fields are non-empty.[1]

4.5 Remediation of Pattern 5: Deterministic Manifest Compilation

Objective: Eliminate runtime indeterminism and globbing risks in context generation.

Architectural Change: The ops/lib/scripts/synthesize.sh script currently resolves context at runtime, which is computationally expensive and potentially hazardous (e.g., if a glob accidentally matches a sensitive file). We will move to a "Build-Time" manifest resolution strategy.[15]

Implementation Specifications:

1. **Source vs. Artifact:** We will distinguish between "Manifest Templates" (in ops/src/manifests/) and "Compiled Manifests" (in ops/lib/manifests/).
   - *Templates:* Allowed to use convenient syntax like globs or comments.
   - *Compiled:* Strict, flat lists of explicit file paths.
2. **Compiler (ops/bin/compile-manifests):** This tool will run offline. It takes the templates, resolves all globs, deduplicates entries, checks for hazards (enforcing the repo/library/ exclusion), and writes the flat lists to ops/lib/manifests/.[13]
3. **Runtime Efficiency:** The synthesize.sh script will be rewritten to simply read the

compiled manifest line-by-line. This reduces the runtime complexity from O(N*M) (recursive directory scanning) to O(1) (reading a file), and guarantees that the context loaded by the AI is exactly what was audited during the compile step.[19]

Table 4.5: Manifest Resolution Strategy

Phase	Activity	Validation	Output
Design	Edit ops/src/manifests/OPS.yaml	Human Review	Source Template
Build	Run ops/bin/compile-manifests	Hazard Check, Glob Expansion	ops/lib/manifests/OPS.md (Flat List)
Runtime	Run ops/bin/context	Existence Check	Context Stream

4.6 Remediation of Pattern 6: Structured Data Separation (Allowlists)

Objective: Remove configuration data from markdown prose to enable reliable, parser-free validation.

Architectural Change: The "Target Files allowlist" is currently a bulleted list inside TASK.md. We will extract this into a dedicated configuration file, storage/dp/active/allowlist.txt.[1]

Implementation Specifications:

1. **Data Extraction:** The ops/bin/draft tool (defined in 4.4) will generate allowlist.txt as a sidecar artifact when creating the DP. This file will contain one file path per line, with no markdown formatting.
2. **Validation Logic:** The tools/lint/dp.sh check for allowlists will be replaced. Instead of parsing markdown AST or regex, it will perform a simple file system check:
   - wc -l < allowlist.txt (Must be > 0)
   - xargs ls < allowlist.txt (All files must exist)
3. **Pointer Reference:** The TASK.md file will no longer contain the list itself. Instead, Section 3.3 will contain a dynamic pointer: "See active allowlist at storage/dp/active/allowlist.txt".[1] This adheres to the "Pointer-First" doctrine while keeping the data in a machine-readable format.[17]

5. Implementation Roadmap and Migration Strategy

Transitioning to this new architecture requires a careful, phased approach to avoid disrupting active operations.

Phase 1: Stabilization (Weeks 1-2)

Focus: Resolving immediate ambiguities without structural upheaval.

1. **Renumber TASK.md:** Update the template to use "Section 3.5" for Closeout.
2. **Update Manual:** Document the concurrency of the closeout phase.
3. **Refactor Prune:** Split ops/bin/prune into clean, archive, reset internally, but keep the prune wrapper for backward compatibility.

Phase 2: Structural Restructuring (Weeks 3-4)

Focus: Physical movement of assets and implementation of CbC controls.

1. **Library Migration:** Execute the move to repo/library/. This will require a massive "search and replace" across all documentation links.
2. **Manifest Compiler:** Implement the compilation step. Freeze ops/lib/manifests/CONTEXT.md as a compiled artifact.
3. **Data Separation:** Introduce allowlist.txt alongside TASK.md.

Phase 3: Immutable Workflow Adoption (Weeks 5-6)

Focus: Locking down the system.

1. **Drafting Tool:** Release ops/bin/draft. Deprecate manual editing of TASK.md structure.
2. **Lint Reduction:** Remove the regex logic from tools/lint/dp.sh. Switch to hash-based structure verification.
3. **Final Cutover:** Update PoT.md to reflect the new physical laws (e.g., recognizing repo/library/ as the new home for canon libraries).

6. Operational Impact and Risk Analysis

6.1 Cognitive Load Reduction

The primary benefit of this architecture is the reduction of cognitive load on the Operator. Currently, the Operator must act as a "human linter," verifying that the automation logic is correct. In the CbC model, structural errors are impossible. The Operator can focus entirely on the *semantic* quality of the work (e.g., "Is this code secure?") rather than the *syntactic* validity of the packet (e.g., "Is the header bold?").[22]

6.2 Determinism and Reproducibility

The shift to compiled manifests and explicit file lists eliminates "works on my machine" errors caused by shell globbing differences across environments. The context loaded by the LLM becomes bit-perfect deterministic, ensuring that every session starts from a known, provable state.[15]

6.3 Risk: Path Breakage

The most significant risk is the breaking of internal links during the Library Migration.

- *Mitigation:* A migration script will be developed to sed replace all docs/library/ references with repo/library/ in a single atomic commit. A specific linter will be temporarily deployed to catch any lingering references.[1]

6.4 Risk: Workflow Friction

Operators accustomed to the "magic" of prune --reset-task (auto-detection) may find the requirement to provide an explicit DP ID cumbersome.

- *Mitigation:* The ops/bin/help curriculum command will be updated to print the exact command string required for the current session, allowing the Operator to copy-paste the correct command easily.[9]

7. Conclusion: The Path to Equilibrium

The "Automation Over Design" analysis revealed a system at war with itself—a constitutional monarchy (PoT) trying to govern a chaotic democracy (LLM prose) using a police state of scripts (Linters). This proposal ends that war.

By embracing **Correct by Construction** principles, we acknowledge that the best way to prevent drift is not to build higher walls, but to build better roads. When the "path of least resistance" is also the "path of correctness," compliance ceases to be a burden and becomes a natural byproduct of the work itself. The transition to immutable structures, physical isolation of hazards, and deterministic data handling will secure Stela's foundation, allowing it to scale its reasoning capabilities without collapsing under the weight of its own administrative complexity.

The proposed architecture transforms the *Policy of Truth* from a set of rules we must *enforce* into a set of physics we simply *inhabit*.

8. Supporting Evidence and References

- **System Analysis:** automation_over_design_analysis.md.[1]
- **Repository State:** dump-full-main-6043fa23.txt.[1]
- **Governance Canon:** PoT.md [1], TASK.md.[1]
- **Design Principles:**

- *Correct by Construction:*.[4]
- *Monolith vs. Granular:*.[9]
- *Prompt Engineering Structures:*.[10]
- *Manifest/Build Safety:*.[13]
- *Structured Data vs. Prose:*.[3]

End of Report. Prepared for DP-OPS-0063 execution.

Works cited

1. OPEN-main-6043fa23.txt
2. Python Markdown Parser - Energent.ai | Convert Markdown to Structured Data, accessed February 14, 2026, https://energent.ai/use-cases/en/python-markdown
3. Understanding Semantic Validation with Structured Outputs - Instructor, accessed February 14, 2026, https://python.useinstructor.com/blog/2025/05/20/understanding-semantic-validation-with-structured-outputs/
4. Correct by Construction - Wikipedia, accessed February 14, 2026, https://en.wikipedia.org/wiki/Correct_by_Construction
5. Correct by Construction: Building Software Right the First Time - DEV Community, accessed February 14, 2026, https://dev.to/copyleftdev/correct-by-construction-building-software-right-the-first-time-2hho
6. Correct By Construction - C2 Wiki, accessed February 14, 2026, https://wiki.c2.com/?CorrectByConstruction
7. Correctness by Construction: A Manifesto for High-Integrity Software - Elsmar.com, accessed February 14, 2026, https://elsmar.com/pdf_files/A%20Manifesto%20for%20High-Integrity%20Software.pdf
8. OWASP LLM Top 10: How it Applies to Code Generation | Learn Article - Sonar, accessed February 14, 2026, https://www.sonarsource.com/resources/library/owasp-llm-code-generation/
9. Breaking down monolith workflows: Modularizing AWS Step Functions workflows | AWS Compute Blog, accessed February 14, 2026, https://aws.amazon.com/blogs/compute/breaking-down-monolith-workflows-modularizing-aws-step-functions-workflows/
10. Overview of prompting strategies | Generative AI on Vertex AI | Google Cloud Documentation, accessed February 14, 2026, https://docs.cloud.google.com/vertex-ai/generative-ai/docs/learn/prompts/prompt-design-strategies
11. 8 Practical Prompt Engineering Tips for Better LLM Apps | by Almog Baku - Medium, accessed February 14, 2026, https://medium.com/data-science/8-practical-prompt-engineering-tips-for-better-llm-apps-430eef9b0950

12. The Ultimate Guide to Prompt Engineering in 2026 | Lakera – Protecting AI teams that disrupt the world., accessed February 14, 2026, https://www.lakera.ai/blog/prompt-engineering-guide
13. The Manifest Format - The Cargo Book - Rust Documentation, accessed February 14, 2026, https://doc.rust-lang.org/cargo/reference/manifest.html
14. Boosting AI Performance: The Power of LLM-Friendly Content in Markdown, accessed February 14, 2026, https://developer.webex.com/blog/boosting-ai-performance-the-power-of-llm-friendly-content-in-markdown
15. Compile-time vs. runtime with C++ templates, accessed February 14, 2026, https://www.tianxiangxiong.com/2025/05/31/compile-time-vs-runtime-with-c++-templates.html
16. Markdown vs JSON? Which one is better for latest LLMs? : r/PromptEngineering - Reddit, accessed February 14, 2026, https://www.reddit.com/r/PromptEngineering/comments/1l2h84j/markdown_vs_json_which_one_is_better_for_latest/
17. Cleaned text vs Markdown: Choosing the Right Output Format for AI | WebcrawlerAPI Blog, accessed February 14, 2026, https://webcrawlerapi.com/blog/cleaned-text-vs-markdown-choosing-the-right-output-format
18. Repository Naming Conventions. Establishing clear and consistent... | by Nurhidayat | Medium, accessed February 14, 2026, https://medium.com/@nur26691/repository-naming-conventions-1065467de776
19. Is it better to specify source files with GLOB or each file individually in CMake?, accessed February 14, 2026, https://stackoverflow.com/questions/1027247/is-it-better-to-specify-source-files-with-glob-or-each-file-individually-in-cmak
20. Folder Structure and Naming Conventions for Project Setup - worldbank.github.io, accessed February 14, 2026, https://worldbank.github.io/template/docs/folders-and-naming.html
21. Structuring a CLI. I love a good CLI, and I like writing... | by Dennis Verweij | Pon.Tech.Talk, accessed February 14, 2026, https://medium.com/pon-tech-talk/structuring-a-cli-22e2492717de
22. UX patterns for CLI tools - Lucas F. Costa, accessed February 14, 2026, https://lucasfcosta.com/blog/ux-patterns-cli-tools
23. danielrosehill/Markdown-Templates-For-LLM-Work - GitHub, accessed February 14, 2026, https://github.com/danielrosehill/Markdown-Templates-For-LLM-Work
24. Design Patterns to Secure LLM Agents In Action - Labs by Reversec, accessed February 14, 2026, https://labs.reversec.com/posts/2025/08/design-patterns-to-secure-llm-agents-in-action
25. eBook | Winning the PCB Trifecta via "Correct by Construction" Design Principles - Cadence, accessed February 14, 2026, https://www5.cadence.com/Correct_by_Construction_ebook.html
26. Small Is beautiful: Granular microservices over monolith applications - PDF

Association, accessed February 14, 2026, https://pdfa.org/small-is-beautiful-granular-microservices-over-monolith-applications/